
How a design becomes an assessed report.

The complete working of the Architecture Intelligence & Design Platform — from the moment a document is uploaded to the moment every clause of your standards has a verdict against it, with a citation.

THE PREMISE

01 What AIDP does

Your standards say every design must be assessed against them. AIDP performs that assessment in minutes instead of days, checks every requirement instead of the ten a reviewer remembers, and cites each finding to the page.

An enterprise standards library typically runs to around 120 standards carrying close to 400 individual, testable requirements. Today the check is a person with a PDF: an architect reads a 60-page vendor design while holding those requirements in their head. It takes days per submission, differs between reviewers, and in practice most requirements are never checked at all.

The standards exist. The enforcement does not. That gap is the product.

What it is not

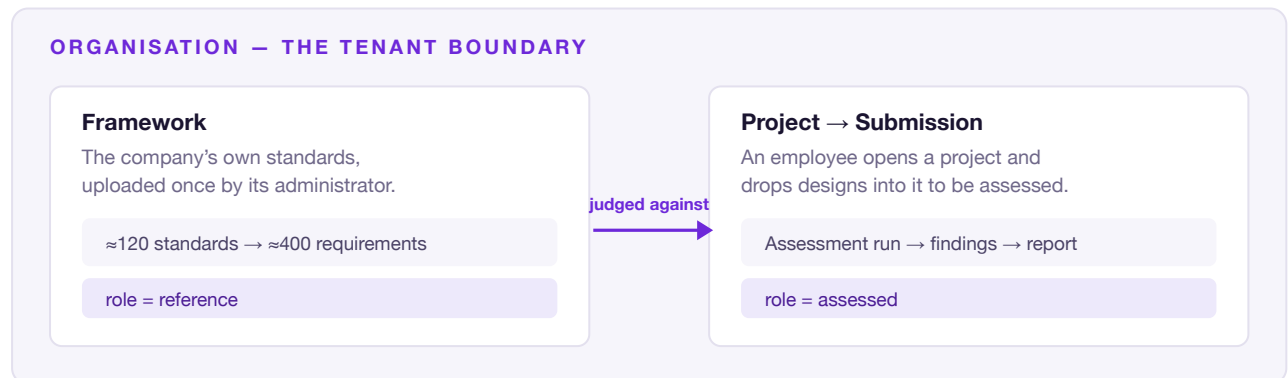
It answers questions, but it is **not a chatbot** — nobody has to ask anything for it to produce value; it runs the whole checklist itself. It stores documents but is not a document store. It does not write policy; it measures against policy.

The distinction drives the engineering. A chatbot is judged on whether the answer reads well. This is judged on whether a “*compliant*” verdict survives an audit.

THE MODEL

02 How the system is arranged

Everything hangs off the **organisation** — one company, one standards library, one hard wall. No customer’s content is reachable from another’s, on any path.



A document is uploaded with one of two **roles**, and the role decides how far it travels through the pipeline. A **reference** document is a standard: it is parsed, chunked, embedded, and stops there — it becomes searchable knowledge. An **assessed** document is a submission: it goes through the same four stages and then continues into the assessment itself.

Who does what

ROLE	CAN DO
Company administrator <i>Your head of architecture</i>	Registers the company. Uploads and owns the standards library. Controls who may use the product — allowing or revoking access per email address, including a bulk import from a spreadsheet. Sees every project in the company and the token spend behind it. Maintains the company profile.
Employee <i>Architects, reviewers</i>	Signs in, creates projects, uploads designs into them, runs assessments and works the findings. Cannot upload or alter the standards library — the rulebook is not editable by the people being measured against it.
Platform operator <i>Us</i>	Sees which companies exist, who their administrators and employees are, and how much the platform is being used. Never sees a customer's documents, findings or reports.

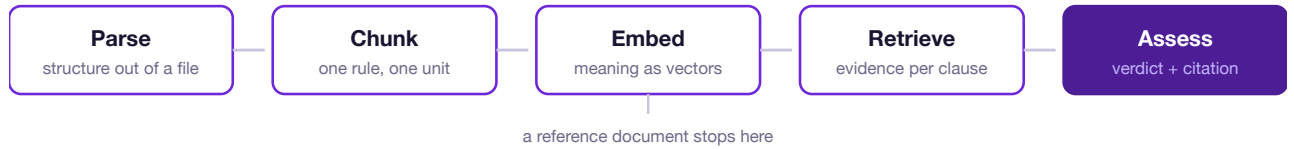
ON ACCESS

Signing in takes an email and a password — nothing else. Which company an address belongs to is decided by the administrator who admitted it, not by the person typing. Admission is re-checked from the roster on *every single request* rather than trusted from the session, so revoking someone takes effect on their very next click.

THE ENGINE

03 The pipeline, end to end

Five stages. Each hands off to the next inside the same database transaction, so a crash never leaves half a document behind.



Parse — a file in, a structured document out

A PDF is a rendering format: whatever structure the author had was destroyed at export. This stage rebuilds it, and the order of operations is deliberate.

- **The table of contents is read first, and used as ground truth.** It gives an independent list of what the body parse *should* find. Reconciling the two afterwards turns silent parse failures into reported ones — the cheapest assurance in the whole system.
- **Headings come from typography, not guesswork.** Every text span carries its font, size, weight and colour; the style that sets the most text is the body style, and anything meaningfully larger or bolder is a heading candidate. A line is promoted only when typography *and* shape agree, so a bold run mid-paragraph cannot split a clause in half.
- **Tables are extracted before figures**, and every cell is bound to its column header with empty cells preserved rather than dropped. This is where a compliance tool is most likely to be confidently wrong: flatten a row with two blank cells and every later value shifts one column left, so “not specified” silently becomes the previous row’s value. Rows split across a page break are rejoined.
- **Diagrams are read, not skipped.** Embedded images and clustered vector drawings are both captured and described by a vision model — an enterprise context diagram holding an entire application landscape often sits behind an empty text layer, and a text-only pipeline extracts nothing from it.
- **Running headers and footers are stripped — but read first.** The sensitivity classification printed in the footer is governance data, not noise, so it is lifted before the strip. The rest is removed so a hundred repetitions of the same line do not pollute every chunk.
- **PowerPoint decks take a shortcut.** A `.pptx` never lost its structure — the file states outright which shape is the title and how deep each bullet sits — so that is read directly instead of inferred.

The atomic unit this all produces is the **clause**: a Statement (the rule), a Rationale (the why), and Requirements (the testable obligations). That shape is what makes the rest of the pipeline possible.

WHEN FORMATTING SAYS NOTHING

A last-resort pass asks a model to read the structure. **The model never sees or returns the document’s text as text** — it receives numbered lines and returns line numbers with a role attached, and we slice our own copy. A model cannot drop a word, truncate a sentence, or “tidy” a requirement it never held. The worst it can do is point at the wrong line, which yields complete, unaltered text grouped imperfectly.

Chunk — turning structure into retrievable units

No fixed-size windows, no character splitting, no “semantic” breakpoint detection. Those techniques exist to *recover* structure from a wall of text. The previous stage already established it, so nothing here re-derives it.

SOURCE	BECOMES
Clause	One chunk, never split. A Requirements bullet severed from its Statement is a useless probe.
Table	One chunk per row with headers bound inline, plus one for the table as a whole.
Figure	The vision model’s description of the diagram.
Section prose	A parent chunk for broader questions — and always kept when a section has nothing else, so it can never become unretrievable.

Every chunk carries its **heading path** and a one-line contextual preamble, so it can be understood — and retrieved — without its neighbours. That heading path is also what a citation is made of.

Embed — meaning stored as numbers

Each chunk is converted into a vector: a list of numbers positioning that passage in meaning-space, so that “*multi-factor authentication is enforced for admin logins*” lands near “*privileged access requires MFA*” even though they share almost no words. Vectors live in Postgres alongside the text, indexed for nearest-neighbour search.

- Only chunks lacking a vector *for the current model* are sent, so a retry after a partial failure costs only what is missing, and a model upgrade rolls out alongside the old one instead of as a big-bang re-embed.
- The passage and the search query are embedded differently, because the two are asymmetric — getting that backwards degrades retrieval quietly rather than loudly.
- Results are **cached against the exact text**, scoped per company. Re-uploading a revised document only pays for the paragraphs that actually changed; a 5% edit costs 5%, not 100%.

04 Retrieval runs backwards

This is the idea the whole product turns on.

In an ordinary search tool, a person types a question and the system finds passages. Here **the standard is the question and the submitted design is the corpus**. Nobody types anything. For each of the ~120 clauses in the framework, the system searches the submitted design for whatever bears on that clause, and classifies what it finds. Run 120 times, that is the whole checklist — automatically.

Two searches, then a fusion

Each clause is searched for twice, because the two methods fail in different places.

Vector search

Finds passages that *mean* the same thing in different words. Strong on paraphrase; soft on exactly the tokens these documents turn on — `TLS 1.2` , `RP0` , `snake_case` , `MFA` .

Keyword search

Postgres full-text. Catches the exact identifiers and acronyms the vectors blur, and misses the paraphrase the vectors catch.

Each returns its own ranked list of up to 32 candidates. They are then merged by **Reciprocal Rank Fusion**: every passage scores $1 / (60 + \text{its rank})$ in each list, and the two scores are added.

Why that formula, in one line

A passage that *both* methods found — even at rank 30 in each — outranks a passage only one of them found at rank 1. Agreement between two independent methods is stronger evidence than confidence from one. The constant 60 keeps the curve flat enough that rank 1 does not simply steamroll everything below it.

The **top 8 passages** go forward. Wide enough that a requirement split across two sections is still visible; narrow enough that the model is not asked to re-read the whole document 120 times over.

WHY RECALL MATTERS MORE HERE THAN ANYWHERE

The assessment has to distinguish “this requirement is genuinely unaddressed” from “the search missed it.” Everything downstream rests on that, which is why two methods run instead of one — and why the fused score is recorded on every finding and used, later, to refuse a verdict retrieval could not support.

05 Assess — reaching a verdict

The clause and its eight retrieved extracts are put to a language model, which is asked one narrow question: *does this design satisfy this clause, using only these extracts?* It returns a verdict, a confidence, a one-sentence rationale, and the specific extract identifiers it relied on.

The instructions it is given are as important as the model itself:

- Any verdict asserting something about the document **must cite at least one extract**. A claim about a document that cites nothing in it was invented.
- **“Absent” is only for extracts plainly about other topics**. If the design seems to touch the subject but the requirement cannot be confirmed, the answer is “needs review” — a wrong “absent” sends someone to fix what already exists; a wrong “covered” ships a gap into production.
- Judge only what the clause requires. The design gets no credit for good practice the clause does not ask for.

Then the answer is checked against its own evidence

The model’s verdict is not taken at face value. Before a finding is written, it passes through guards that demote anything the evidence does not support — and each demotion states why, so the reviewer sees the reasoning rather than an unexplained shrug.

IF ...	THEN
A verdict claims something about the design but cites no extract	Needs review
A decisive verdict rests only on a diagram description — a model’s reading of an image, not text from the page	Needs review
“Absent” is reached on top of weak retrieval — nothing relevant was found, which is not the same as nothing existing	Needs review
“Absent” is reached with confidence below 55%	Needs review

These guards are the difference between a tool a reviewer can trust and one that is confidently wrong at scale. “Needs review” costs a human two minutes. The alternatives cost considerably more.

THE OUTPUT

06 The report

Every clause in the framework produces exactly one finding, carrying its verdict, the reasoning, and the passages it was based on — each with its heading path and page number in the submitted document.

VERDICT	MEANS
Contradicts	The design states something the standard forbids, or forbids something it requires.
Absent	The design does not address this requirement at all.
Partial	Addressed, but a requirement is left unmet.
Needs review	The evidence was inconclusive. A person has to decide.
Covered	Every requirement in the clause is met.

A report **opens on what needs attention** — everything except *Covered* — because that is the queue a reviewer actually works. The covered findings are folded away rather than discarded, with their count still on screen and one click to see them: an audit asks precisely for the evidence that a clause was checked and passed.

What the architect's job becomes

Confirming and overriding findings rather than hunting for them. The reviewer stops being a search engine. Each finding records who reviewed it and what they concluded, so the report is an audit trail rather than a snapshot.

A finding, in shape

Contradicts Solution Architecture §3.5 — Statelessness

Application services must not hold session state in local memory; state belongs in an external store so instances remain interchangeable.

“The design places session state in application memory, which the clause explicitly forbids.”

Cited from the submission — **§4.2 Session Management, page 17**